PWA en Rails

Tu app Rails, instalable en cualquier dispositivo

Sobre mí

- **Juan Vásquez** (@JuanVqz)
- Senior Software Engineer
- Laburo en OmbuLabs (FastRuby.io)
- Soy de Oaxaca, México 🇲🇽
- Pasión actual: Crochet 🧶

Sobre esta charla

- Qué es una PWA y qué necesita
- Por qué Rails decidió incluir soporte nativo
- Demo en vivo
- Qué viene: Action Notifier y Web Push

Qué es una PWA?

Progressive Web App = Una app web que se comporta como nativa.

Tres pilares:

1. **Instalable** -- Se agrega a la pantalla de inicio
2. **Confiable** -- Carga incluso sin conexión (o con red lenta)
3. **Capaz** -- push notifications, acceso a cámara, GPS, etc.

> *No es un framework, es un conjunto de APIs del navegador + un patrón de arquitectura.*

Qué necesita una web para ser PWA?

Requisito	Qué es	Obligatorio?
Web App Manifest	JSON que describe la app (nombre, iconos, colores)	Sí
Service Worker	Script que intercepta requests del navegador	No, agrega superpoderes
HTTPS	Conexión segura (requerido para service workers)	Solo para Service Worker

Sólo con el manifest ya podés hacer tu app **instalable**.

Por qué HTTPS para service workers?

Un service worker puede **interceptar todos los requests** de tu app.

Sin HTTPS, un atacante podría:

- Inyectar un service worker malicioso (man-in-the-middle)
- Robar datos, redirigir requests, modificar respuestas

Qué es un Service Worker?

Un archivo JavaScript que corre **en un hilo separado** del navegador.

```
Browser                    Service Worker                  Red
   │                                │                        │
   │── GET /orders ────────────────>│                        │
                                    │── (decide) ────────────>│
                                    <── response ────────────│
   <── response ────────────────────│                        │
```

- No tiene acceso al DOM
- Actúa como **proxy** entre el browser y la red
- Puede cachear respuestas, servir offline, recibir push notifications
- Tiene un **ciclo de vida** propio: install → activate → fetch

Por qué Rails incluyó soporte PWA?

Rails quiere ser una alternativa completa a las apps nativas:

- Hotwire para interactividad.
- PWA para instalabilidad.
- Action Notifier (futuro) para push notifications.

> *Rails 7.2* ***PR #50528*** *de DHH.*

Qué genera Rails?

```
rails new rubysur
```

Tres archivos relacionados a PWA:

```
config/routes.rb                    # Rutas
app/views/pwa/manifest.json.erb     # Web App Manifest
app/views/pwa/service-worker.js     # Service Worker (comentado)
```

Importante: todo viene *desactivado* por defecto. Es opt-in.

Demo 1: Hacer tu página instalable

- Rutas
- Meta Tags
- Manifest

Demo 1: Resultado

Con solo esto, tu app ya es **instalable**:

- Chrome muestra el ícono de instalar en la barra de direcciones
- En móvil, "Agregar a pantalla de inicio" funciona
- La app se abre en modo standalone (sin barra del browser)

DevTools → Application → Manifest para verificar.

> *3 archivos, 5 líneas de config. ~10 minutos.*

Demo 2: Página offline cuando no hay red

- Registro del service worker
- La página offline
- El Service Worker sirve la pagina offline

Demo 2: Resultado

1. Cargar la app con red → el SW se instala y cachea `offline.html`
2. Cortar la red (DevTools → Network → Offline)
3. Navegar → en vez del dinosaurio, se ve nuestra página

DevTools → Application → Service Workers para verificar.

> *~25 líneas de JS. El usuario ve tu marca, no un error del browser.*

Demo 3: Estrategias de Caché

Assets instantáneos + HTML fresco

Estrategias de caché: resumen rápido

Estrategia	Idea	Ideal para
Cache First	Cache, si no hay, red	Assets (CSS, JS, iconos)
Network First	Red, si falla, caché	HTML dinámico
Stale While Revalidate	Cache + actualizar en background	Datos que cambian poco

> *La clave: no usar una sola estrategia.* ***Mezclar según el tipo de request.***

Demo 3: Resultado

1. Primera visita: todo se descarga de la red
2. Segunda visita: los assets cargan **del cache** (instantáneo)
3. El HTML siempre viene fresco de la red
4. Sin red: se muestra la página offline

DevTools → Application → Cache Storage para ver qué se cacheó.

> *Assets rápidos + datos frescos. Lo mejor de ambos mundos para una app Rails.*

Demo 4: PWA + Multi-Store

Cada tienda, su propia PWA

Nuestra app es multi-store

Cada tienda es un tenant con su propio subdominio:

```
cafecito.maystore.app        → Tienda "Cafecito"
la-esquina.maystore.app      → Tienda "La Esquina"
www.maystore.app             → Landing page publica
```

La pregunta: **afecta esto a la PWA?**

Respuesta corta: **si, pero a nuestro favor.**

Service Worker y orígenes

Un service worker controla las páginas **de su mismo origen** (protocolo + dominio + puerto).

```
cafecito.maystore.app      → origen A → service worker A
la-esquina.maystore.app    → origen B → service worker B
```

Cada subdominio es un **origen distinto**:

- Cada tienda tiene su **propio** service worker
- Cada tienda tiene su **propio** caché
- Cada tienda puede instalarse como **su propia PWA**

Demo 4: Cache nombrado por store

```
// app/views/pwa/service-worker.js.erb
const CACHE_NAME = "may-store-<%= current_store&.subdomain || 'public' %>-v1";
```

Resultado en DevTools:

```
cafecito.maystore.app       → cache "may-store-cafecito-v1"
la-esquina.maystore.app     → cache "may-store-la-esquina-v1"
www.maystore.app            → cache "may-store-public-v1"
```

> *Aunque el browser ya aisla por origen, nombrar el caché por store facilita depurar.*

Cuidado: el PwaController de Rails

`Rails::PwaController` hereda de `Rails::ApplicationController`, **no** del tuyo.

```
# Tu ApplicationController setea Current.store:
class ApplicationController < ActionController::Base
  before_action :set_current_store

  def set_current_store
    Current.store = Store.find_by!(subdomain: request.subdomain)
  end
end

# Pero Rails::PwaController NO hereda de ahi.
# → Current.store es nil → el manifest no personaliza
```

Fix: definir `current_store` como helper

Agregar un helper que use `Current.store` con fallback, disponible en **todas** las vistas, incluidas las del PwaController:

```
# app/helpers/application_helper.rb
module ApplicationHelper
  def current_store
    Current.store || Store.find_by(subdomain: request.subdomain)
  end
end
```

Si `Current.store` ya está seteado (desde tu controller), lo usa. Si no (PwaController), hace la query.

> *Alternativa: crear tu propio PwaController que herede de `ApplicationController`.*

Demo 4: Resultado

Cada tienda:

- Se instala como PWA independiente con su nombre y logo
- Tiene su propio service worker y cache
- Funciona offline de forma aislada

En DevTools: cambiar entre subdominios y verificar que cada uno tiene su manifest y caché separados.

Cuánto esfuerzo es?

Qué	Archivos	Lineas de codigo
Manifest	1	~20
Service Worker	1	~30
Rutas	2 lineas en routes.rb	2
Meta tags	3 lineas en layout	3
Total	**3 archivos + 5 lineas**	**~55**

> *En 10 minutos tu app Rails es instalable. En 30 minutos tiene offline básico.*

Tips para producción

Iconos: generá múltiples tamaños (192x192, 512x512) con realfavicongenerator.net

Cache busting: cambiar `CACHE_NAME` para invalidar

```
const CACHE_NAME = "may-store-v2" // Cambiar para forzar nuevo cache
```

`scope` **en el manifest:** define qué URLs "pertenecen" a la PWA. Si tu app vive en `/admin/`, usá `"scope": "/admin/"` y `"start_url": "/admin/"`. Es un campo del manifest, no del service worker.

HTTPS: obligatorio en producción. `localhost` funciona sin SSL.

Qué viene: Action Notifier

Rails está trabajando en **Action Notifier** -- un framework para enviar notificaciones Web Push directamente desde Rails.

```
# Futuro (conceptual)
class OrderNotifier < ApplicationNotifier
  def ready(order)
    notify order.spot,
      title: "Pedido listo",
      body: "El pedido #{order.code} está listo para entregar"
  end
end
```

> *Mismo patrón que Action Mailer, pero para push notifications.*

Recursos

- **PR original:** rails/rails#50528
- **Rails World 2024:** Emmanuel Hayford -- "PWA for Rails developers"
- **Joy of Rails:** Add your Rails app to the Home Screen
- **SupeRails #166:** PWA in Rails 8
- **MDN Web Docs:** Progressive Web Apps
- **web.dev:** Learn PWA

Contribuciones

- https://github.com/rails/rails/pull/57184
- https://github.com/mdn/translated-content/pull/35207
- https://github.com/mdn/translated-content/pull/35016

Preguntas?

Gracias!

RubySur - 13 de abril de 2026

- Slides: `github.com/JuanVqz/talks/pwa_en_rails`
- PRs: `github.com/JuanVqz/may_store`
- Gists: `gist.github.com/JuanVqz`
- Video: `youtube.com/watch?v=ppxalpIKpGg`